

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Andika Fathurrahman Achiral - 5024221065

2026

1 Langkah-Langkah Percobaan

Pada Modul 1 ini dilakukan lima rangkaian percobaan untuk mengenal arsitektur serta lingkungan pengembangan dua jenis mikrokontroler, yaitu STM32 Blackpill dan ESP32 DevKit, meliputi konfigurasi *digital output*, *digital input* dengan resistor *pull-up*/*pull-down*, penanganan *debouncing* pada tombol mekanik, serta pembacaan *analog input* melalui peripheral *Analog-to-Digital Converter* (ADC).

1.1 Alat dan Bahan

Seluruh perangkat keras dan perangkat lunak yang dipersiapkan serta digunakan selama pelaksanaan praktikum Modul 1 dirinci pada Tabel 1.

Tabel 1: Daftar alat dan bahan praktikum Modul 1

No	Nama Komponen	Spesifikasi / Keterangan	Jumlah
1	Board STM32 Blackpill	STM32F401CCU6	1
2	ST-Link V2	Programmer/debugger eksternal via antarmuka SWD	1
3	Board ESP32 DevKit	ESP32 DevKit v1	1
4	Kabel jumper female-female	Penghubung ST-Link V2 ke STM32 Blackpill	4
5	Kabel USB Micro / USB-C	Penghubung ST-Link dan ESP32 ke komputer	2
6	Breadboard	830 titik	1
7	Kabel jumper male-male	Penghubung rangkaian pada breadboard	Secukupnya
8	LED + resistor $220\,\Omega$	Uji Blink ESP32 (apabila tidak terdapat LED onboard)	1
9	Pushbutton (tactile)	Percobaan 3 (*pull-up*/*pull-down*) dan Percobaan 4 (*debouncing*)	2
10	Resistor $10\,\mathrm{k}\Omega$	*Pull-up*/*pull-down* eksternal (Percobaan 3) dan pembagi tegangan LDR (Percobaan 5)	3
11	LDR	*Light Dependent Resistor* (Percobaan 5)	1
12	Laptop / PC	Terpasang Visual Studio Code dan ekstensi PlatformIO	1

1.2 Persiapan Lingkungan Pengembangan dan Proyek PlatformIO

Sebelum melaksanakan seluruh percobaan, dilakukan tahap persiapan perangkat lunak dan struktur proyek sebagai berikut:

1. Memastikan Visual Studio Code, ekstensi PlatformIO IDE, serta driver USB yang diperlukan (driver ST-Link untuk STM32 dan driver USB-to-UART CP2102/CH340 untuk ESP32) telah terpasang dengan baik pada komputer.

2. Menyiapkan board STM32 Blackpill beserta programmer ST-Link V2 dan board ESP32 DevKit dalam keadaan belum terhubung ke port USB komputer.

3. Membuat **satu** proyek PlatformIO baru melalui menu *PlatformIO Home* (*New Project*) dengan nama proyek `modul1-praktikum` yang digunakan secara terpadu untuk seluruh Percobaan 1 hingga Percobaan 5.

4. Menyesuaikan isi berkas konfigurasi `platformio.ini` serta berkas kode sumber utama (`src/main.cpp` atau `src/main.c`) pada proyek yang sama setiap kali berpindah percobaan, kemudian menunggu proses *refresh*/*re-index* PlatformIO pada *status bar* Visual Studio Code selesai sebelum menjalankan perintah **Build** maupun **Upload**.

1.3 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Percobaan pertama bertujuan menguji proses pemrograman dasar (*flashing*) pada board STM32F401CCU6 Blackpill menggunakan framework Arduino (STM32duino) dan programmer ST-Link V2 melalui antarmuka *Serial Wire Debug* (SWD) dengan memanfaatkan LED onboard pada pin `PC13`.

Konfigurasi `platformio.ini`

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

Prosedur Pelaksanaan

1. Membuka proyek `modul1-praktikum` dan mengubah isi berkas `platformio.ini` sesuai konfigurasi board `blackpill_f401cc` dengan framework `arduino` dan protokol unggah `stlink`, lalu menunggu proses *re-index* PlatformIO selesai.
2. Menghubungkan pin programmer ST-Link V2 ke header SWD pada STM32 Blackpill menggunakan empat kabel jumper *female-female* sesuai pemetaan pada Tabel 2, serta memastikan jalur tegangan referensi berada pada 3.3 V.
3. Menghubungkan ST-Link V2 ke port USB komputer.
4. Menuliskan kode program Blink ke dalam berkas `src/main.cpp`.
5. Menekan tombol **Build** untuk mengompilasi program, kemudian menekan tombol **Upload** untuk mengunggah program ke mikrokontroler STM32 Blackpill.
6. Mengamati nyala LED onboard pada pin `PC13` yang berkedip secara periodik (menyala selama 500 ms saat berlogika `LOW` karena bersifat *active-low*, dan padam selama 500 ms saat berlogika `HIGH`).

Tabel 2: Pemetaan koneksi pin ST-Link V2 ke STM32 Blackpill

Pin ST-Link V2	Pin STM32 Blackpill	Fungsi
SWDIO	SWDIO	Jalur data serial debug/pemrograman
SWCLK	SWCLK	Jalur sinyal clock debug/pemrograman
GND	GND	Referensi ground bersama
3.3V	3V3	Tegangan catu/referensi target board (3.3 V)

Kode Program (`src/main.cpp`)

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

1.4 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Percobaan kedua bertujuan mengenalkan arsitektur pemrograman native ESP32 menggunakan framework ESP-IDF berbasis FreeRTOS melalui fungsi utama `app_main()` untuk mengendalikan LED onboard pada `GPIO_NUM_2`.

Konfigurasi `platformio.ini`

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

Prosedur Pelaksanaan

1. Melepaskan koneksi ST-Link V2 dari komputer, lalu mengubah isi berkas `platformio.ini` menggunakan konfigurasi board `esp32dev` dengan framework `espidf` dan menunggu proses konfigurasi ulang PlatformIO selesai.
2. Menghubungkan board ESP32 DevKit v1 ke port USB komputer menggunakan kabel USB, serta memastikan port serial terdeteksi oleh sistem operasi.
3. Menghapus berkas `src/main.cpp` pada direktori proyek, kemudian membuat berkas sumber baru bernama `src/main.c`.
4. Menuliskan kode program Blink berbasis ESP-IDF dan FreeRTOS pada `src/main.c`.
5. Menjalankan proses **Build** dan **Upload** ke board ESP32 DevKit.
6. Mengamati kedipan LED onboard ESP32 (`GPIO 2`) dengan interval waktu tunda 500 ms menggunakan fungsi `vTaskDelay(pdMS_TO_TICKS(500))`, serta mencatat perbedaan struktur pemrograman antara ESP-IDF dan Arduino.

Kode Program (`src/main.c`)

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

1.5 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Percobaan ketiga dilakukan untuk mengamati dan membandingkan pembacaan masukan digital dari dua buah *pushbutton* menggunakan konfigurasi resistor *pull-up* dan *pull-down*, baik yang dirangkai secara eksternal menggunakan resistor fisik $10\,\text{k}\Omega$ (Bagian A) maupun yang diaktifkan secara internal melalui perangkat lunak pada ESP32 (Bagian B).

Konfigurasi `platformio.ini`

```ini
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

Prosedur Pelaksanaan Bagian A (Pull-up dan Pull-down Eksternal)

1. Mengubah kembali konfigurasi `platformio.ini` ke framework `arduino`, menghapus berkas `src/main.c`, dan membuat kembali berkas `src/main.cpp`.
2. Merangkai dua buah *pushbutton* dan dua buah resistor $10\,\text{k}\Omega$ pada breadboard yang terhubung ke pin `GPIO 32` dan `GPIO 33` ESP32 sesuai pemetaan rangkaian eksternal pada Tabel 3.
3. Menuliskan **Kode Program A (Eksternal)** pada berkas `src/main.cpp`, kemudian melakukan **Build** dan **Upload**.
4. Membuka *Serial Monitor* dengan *baud rate* $115200\,\text{bps}$, menekan masing-masing tombol secara bergantian, dan mencatat status logika yang terbaca.

Tabel 3: Skema pengkabelan Bagian A (*pull-up* dan *pull-down* eksternal)

Komponen	Pin ESP32	Keterangan Koneksi
Pushbutton 1 (*pull-up* eksternal)	GPIO 32	Satu kaki ke GPIO 32 dan ke $3.3\,\text{V}$ melalui resistor $10\,\text{k}\Omega$; kaki lainnya ke GND.
Pushbutton 2 (*pull-down* eksternal)	GPIO 33	Satu kaki ke GPIO 33 dan ke GND melalui resistor $10\,\text{k}\Omega$; kaki lainnya ke $3.3\,\text{V}$.

Kode Program A: Pull-up dan Pull-down Eksternal (`src/main.cpp`)

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal, bukan
    INPUT_PULLUP internal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

Prosedur Pelaksanaan Bagian B (Pull-up dan Pull-down Internal)

1. Melepaskan kedua resistor eksternal $10\,\text{k}\Omega$ dari breadboard dengan tetap mempertahankan posisi kedua *pushbutton* pada pasangan kaki kontak yang tepat sesuai Tabel 4.
2. Mengganti isi berkas `src/main.cpp` dengan **Kode Program B (Internal)** yang mengaktifkan mode `INPUT_PULLUP` pada `GPIO 32` dan `INPUT_PULLDOWN` pada `GPIO 33`.
3. Melakukan **Build** dan **Upload**, lalu membuka *Serial Monitor* ($115200\,\text{bps}$) dan menguji penekanan kedua tombol.
4. Membandingkan keluaran status penekanan tombol antara konfigurasi eksternal (Bagian A) dan konfigurasi internal (Bagian B).

Kode Program B: Pull-up dan Pull-down Internal (`src/main.cpp`)

```
#include <Arduino.h>

```

Tabel 4: Skema pengkabelan Bagian B (*pull-up* dan *pull-down* internal)

Komponen	Pin ESP32	Keterangan Koneksi
Pushbutton 1 (*pull-up* internal)	GPIO 32	Satu kaki ke GPIO 32, kaki lainnya ke GND; tanpa resistor eksternal (`INPUT_PULLUP`).
Pushbutton 2 (*pull-down* internal)	GPIO 33	Satu kaki ke GPIO 33, kaki lainnya ke 3.3 V; tanpa resistor eksternal (`INPUT_PULLDOWN`).

```
3  #define BUTTON_PULLUP_PIN 32   // pull-up internal, pin sama seperti Kode Program A
4  #define BUTTON_PULLDOWN_PIN 33 // pull-down internal, pin sama seperti Kode Program A
5
6  void setup() {
7    Serial.begin(115200);
8    pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32, tanpa
       resistor eksternal
9    pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32, tanpa
       resistor eksternal
10 }
11
12 void loop() {
13   int stateUp = digitalRead(BUTTON_PULLUP_PIN);
14   int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);
15
16   Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
17                 stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
18                 stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
19   delay(200);
20 }
```

1.6 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Percobaan keempat bertujuan mengamati fenomena *bouncing* pada kontak mekanik *pushbutton* serta menguji efektivitas algoritma *software debouncing* berbasis pewaktu non-blocking `millis()` dengan membandingkan dua pencacah (`counterNoDebounce` dan `counterWithDebounce`).

Prosedur Pelaksanaan

1. Mempertahankan konfigurasi `platformio.ini` dari Percobaan 3 (board `esp32dev`, framework `arduino`).
2. Merangkai satu buah *pushbutton* pada breadboard dengan menghubungkan satu kakinya ke pin `GPIO 25` ESP32 dan kaki pasangannya ke jalur `GND` (memanfaatkan resistor `INPUT_PULLUP` internal).
3. Menuliskan kode program perbandingan pencacah tanpa dan dengan *debouncing* pada berkas `src/main.cpp` dengan nilai awal `DEBOUNCE_DELAY` sebesar 50 ms, lalu menjalankan **Build** dan **Upload**.

4. Membuka *Serial Monitor* (115200 bps) dan menekan tombol satu kali dengan variasi kecepatan serta tekanan (tekanan cepat dan tegas, maupun tekanan perlahan dan ragu-ragu), kemudian mencatat perbandingan nilai `counterNoDebounce` dan `counterWithDebounce`.

5. Mengubah nilai konstanta `DEBOUNCE_DELAY` menjadi 10 ms, melakukan **Build** dan **Upload**, lalu menguji kembali penekanan tombol dan mencatat hasilnya.

6. Mengubah nilai konstanta `DEBOUNCE_DELAY` menjadi 200 ms, melakukan **Build** dan **Upload**, lalu menekan tombol secara cepat dan berulang kali untuk mengamati kestabilan pembacaan serta kecepatan respons tombol terhadap ketiga variasi waktu tunda (10 ms, 50 ms, dan 200 ms).

Kode Program (`src/main.cpp`)

```cpp
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  int reading = digitalRead(BUTTON_PIN);

  // ===== Tanpa Debouncing =====
  // Langsung dihitung setiap transisi HIGH->LOW, tanpa validasi waktu
  if (reading == LOW && rawLastState == HIGH) {
    counterNoDebounce++;
  }
  rawLastState = reading;

  // ===== Dengan Debouncing =====
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
```

```
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai secara bersamaan =====
  static unsigned long lastPrintedNoDebounce = 0, lastPrintedWithDebounce = 0;
  if (counterNoDebounce != lastPrintedNoDebounce || counterWithDebounce !=
    lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n", counterNoDebounce,
     counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}
```

1.7 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Percobaan kelima bertujuan mengimplementasikan mode akses GPIO ketiga, yaitu *analog input*, menggunakan peripheral ADC 12-bit pada kanal ADC1 (`GPIO 34`) ESP32 untuk membaca perubahan tegangan pada rangkaian pembagi tegangan sensor cahaya *Light Dependent Resistor* (LDR).

Prosedur Pelaksanaan

1. Mempertahankan konfigurasi `platformio.ini` pada board `esp32dev` dengan framework `arduino`.
2. Merangkai sensor LDR dan resistor tetap $10\,\text{k}\Omega$ secara seri sebagai rangkaian pembagi tegangan pada breadboard sesuai pemetaan pada Tabel 5, dengan titik tengah kedua komponen dihubungkan ke pin `GPIO 34` ESP32.
3. Menuliskan kode program pembacaan nilai mentah ADC (`analogRead(34)`) beserta konversinya ke satuan tegangan (0–$3.3\,\text{V}$) pada berkas `src/main.cpp`.
4. Menjalankan **Build** dan **Upload**, lalu membuka *Serial Monitor* ($115200\,\text{bps}$).
5. Memberikan variasi intensitas cahaya pada permukaan LDR dengan cara menutup LDR menggunakan tangan (kondisi gelap), membiarkannya pada pencahayaan ruangan normal, serta menyinarinya menggunakan sumber cahaya lampu (kondisi terang), kemudian mencatat perubahan nilai mentah ADC (0–4095) dan nilai tegangan keluarannya.

Tabel 5: Skema pengkabelan pembagi tegangan sensor LDR pada Percobaan 5

Komponen	Pin ESP32	Keterangan Koneksi
LDR + Resistor $10\,\text{k}\Omega$	GPIO 34 (ADC1_CH6)	Satu kaki LDR ke $3.3\,\text{V}$; kaki LDR lainnya dihubungkan ke titik simpul `GPIO 34` dan ke GND melalui resistor $10\,\text{k}\Omega$.

Kode Program (`src/main.cpp`)

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

2 Analisis Hasil Percobaan

Berisi pembahasan terhadap hasil yang diperoleh selama praktikum. Analisis dilakukan dengan membandingkan hasil percobaan terhadap teori, serta mengidentifikasi faktor-faktor yang memengaruhi hasil, seperti kesalahan alat atau langkah percobaan. Tujuan bagian ini adalah untuk mengevaluasi keberhasilan percobaan dan memastikan pemahaman praktikan terkait praktikum yang telah dilaksanakan.

3 Hasil Tugas Modul

Bagian ini memuat hasil dari pengerjaan tugas tambahan yang diberikan dalam modul praktikum serta penjelasan dari jawaban tersebut.

4 Kesimpulan

Kesimpulan berisi ringkasan dari hasil praktikum dan hal-hal penting yang didapatkan. Bagian ini menjawab tujuan praktikum, mencantumkan hasil yang sesuai atau tidak sesuai dengan teori, serta pembelajaran yang diperoleh oleh praktikan.

5 Lampiran

5.1 Dokumentasi saat praktikum

Menampilkan foto selama pelaksanaan praktikum. Dokumentasi meliputi foto alat yng digunakan dan foto praktikan saat praktikum. Tujuannya sebagai bukti telah dilakukan kegiatan praktikum.